

Goal Setting & Tracking Portal — Complete Step-by-Step Development Guide

Based on: AtomQuest Hackathon 1.0 Problem Statement

1. Project Overview

We are building a complete enterprise-style Goal Setting & Tracking Portal.

The portal will support:

- Employee Goal Creation
 - Manager Approval Workflow
 - Quarterly Achievement Tracking
 - Check-in Comments
 - Dashboard & Reporting
 - Audit Logs
 - Role-based Access
-

2. Recommended Technology Stack

Frontend

- Next.js 15
- Tailwind CSS
- shadcn/ui
- Axios
- React Hook Form

Backend

- Flask
- Flask JWT Extended
- SQLAlchemy
- Flask-CORS

Database

- PostgreSQL

Deployment

- Frontend → Vercel

- Backend → Render
 - Database → Supabase PostgreSQL
-

3. Final Architecture

```
graph TD;
  A[Frontend (Next.js)] --> B[REST API (Flask)];
  B --> C[PostgreSQL Database];
```

4. Main Features

Employee

- Login
- Create goals
- Submit goals
- Quarterly achievement updates
- View locked goals

Manager

- View team goals
- Approve/reject goals
- Edit targets and weightage
- Add check-in comments

Admin

- Unlock goals
 - Manage cycles
 - View reports
 - View audit logs
-

5. Project Setup

STEP 1 — Create Project Folder

```
mkdir goal-tracking-portal  
cd goal-tracking-portal
```

STEP 2 — Create Frontend

Create Next.js App

```
npx create-next-app@latest frontend
```

Choose:

- TypeScript → Yes
 - Tailwind → Yes
 - App Router → Yes
 - ESLint → Yes
-

STEP 3 — Create Backend

```
mkdir backend  
cd backend
```

STEP 4 — Create Virtual Environment

Windows

```
python -m venv venv  
venv\Scripts\activate
```

Mac/Linux

```
python3 -m venv venv  
source venv/bin/activate
```

STEP 5 — Install Backend Dependencies

```
pip install flask
pip install flask-cors
pip install flask-sqlalchemy
pip install flask-jwt-extended
pip install pycopg2-binary
pip install python-dotenv
pip install bcrypt
```

STEP 6 — Create Backend Folder Structure

```
backend/
├── app.py
├── config.py
├── .env
├── models/
├── routes/
├── services/
├── utils/
└── requirements.txt
```

STEP 7 — Freeze Requirements

```
pip freeze > requirements.txt
```

6. Database Design

STEP 8 — Create Database Tables

Users Table

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100) UNIQUE,
```

```
password VARCHAR(255),  
role VARCHAR(20),  
manager_id INTEGER  
);
```

Goals Table

```
CREATE TABLE goals (  
  id SERIAL PRIMARY KEY,  
  employee_id INTEGER,  
  thrust_area VARCHAR(100),  
  title VARCHAR(255),  
  description TEXT,  
  uom VARCHAR(50),  
  target_value FLOAT,  
  achievement FLOAT DEFAULT 0,  
  weightage FLOAT,  
  status VARCHAR(50),  
  quarter VARCHAR(20),  
  is_locked BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Check-ins Table

```
CREATE TABLE checkins (  
  id SERIAL PRIMARY KEY,  
  goal_id INTEGER,  
  quarter VARCHAR(20),  
  achievement FLOAT,  
  employee_comment TEXT,  
  manager_comment TEXT,  
  status VARCHAR(50),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Audit Logs Table

```
CREATE TABLE audit_logs (  
  id SERIAL PRIMARY KEY,  
  user_id INTEGER,  
  goal_id INTEGER,
```

```
    action VARCHAR(255),
    old_value TEXT,
    new_value TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

7. Flask Backend Development

STEP 9 — Create Flask App

app.py

```
from flask import Flask
from flask_cors import CORS
from flask_sqlalchemy import SQLAlchemy
from flask_jwt_extended import JWTManager
from dotenv import load_dotenv
import os

load_dotenv()

app = Flask(__name__)

app.config['SQLALCHEMY_DATABASE_URI'] = os.getenv('DATABASE_URL')
app.config['JWT_SECRET_KEY'] = os.getenv('JWT_SECRET_KEY')

CORS(app)

db = SQLAlchemy(app)
jwt = JWTManager(app)

@app.route('/')
def home():
    return {
        'message': 'Goal Tracking API Running'
    }

if __name__ == '__main__':
    app.run(debug=True)
```

STEP 10 — Create Environment Variables

.env

```
DATABASE_URL=postgresql://postgres:password@localhost/goalsdb
JWT_SECRET_KEY=supersecretkey
```

STEP 11 — Run Backend

```
python app.py
```

Open:

```
http://127.0.0.1:5000
```

8. Authentication System

STEP 12 — Create User Model

models/user_model.py

```
from app import db

class User(db.Model):
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100))
    email = db.Column(db.String(100), unique=True)
    password = db.Column(db.String(255))
    role = db.Column(db.String(20))
    manager_id = db.Column(db.Integer)
```

STEP 13 — Create Auth Routes

routes/auth_routes.py

```
from flask import Blueprint, request
from flask_jwt_extended import create_access_token
from models.user_model import User
from app import db
import bcrypt

auth_bp = Blueprint('auth', __name__)

@auth_bp.route('/register', methods=['POST'])
def register():
    data = request.json

    hashed = bcrypt.hashpw(
        data['password'].encode('utf-8'),
        bcrypt.gensalt()
    )

    user = User(
        name=data['name'],
        email=data['email'],
        password=hashed.decode('utf-8'),
        role=data['role']
    )

    db.session.add(user)
    db.session.commit()

    return {'message': 'User registered'}
```

STEP 14 — Create Login API

```
@auth_bp.route('/login', methods=['POST'])
def login():
    data = request.json

    user = User.query.filter_by(email=data['email']).first()

    if not user:
        return {'error': 'User not found'}, 404

    valid = bcrypt.checkpw(
        data['password'].encode('utf-8'),
```



```

        user.password.encode('utf-8')
    )

    if not valid:
        return {'error': 'Invalid password'}, 401

    token = create_access_token(identity={
        'id': user.id,
        'role': user.role
    })

    return {
        'token': token,
        'role': user.role
    }

```

9. Goal Management Module

STEP 15 — Create Goal Model

models/goal_model.py

```

from app import db

class Goal(db.Model):
    __tablename__ = 'goals'

    id = db.Column(db.Integer, primary_key=True)
    employee_id = db.Column(db.Integer)
    thrust_area = db.Column(db.String(100))
    title = db.Column(db.String(255))
    description = db.Column(db.Text)
    uom = db.Column(db.String(50))
    target_value = db.Column(db.Float)
    achievement = db.Column(db.Float, default=0)
    weightage = db.Column(db.Float)
    status = db.Column(db.String(50))
    quarter = db.Column(db.String(20))
    is_locked = db.Column(db.Boolean, default=False)

```

STEP 16 — Validation Rules

The problem statement requires:

- Total weightage = 100%
- Min weightage = 10%
- Max goals = 8

Validation Logic

```
if len(goals) > 8:
    return {'error': 'Maximum 8 goals allowed'}

for goal in goals:
    if goal['weightage'] < 10:
        return {'error': 'Minimum weightage is 10%'}

total = sum(goal['weightage'] for goal in goals)

if total != 100:
    return {'error': 'Total weightage must equal 100%'}
```

STEP 17 — Create Goal API

routes/goal_routes.py

```
from flask import Blueprint, request
from models.goal_model import Goal
from app import db

goal_bp = Blueprint('goals', __name__)

@goal_bp.route('/goals', methods=['POST'])
def create_goal():
    data = request.json

    goal = Goal(
        employee_id=data['employee_id'],
        thrust_area=data['thrust_area'],
        title=data['title'],
        description=data['description'],
        uom=data['uom'],
        target_value=data['target_value'],
        weightage=data['weightage'],
```

```
        status='Not Started',
        quarter='Q1'
    )

    db.session.add(goal)
    db.session.commit()

    return {'message': 'Goal created'}
```

10. Manager Approval Workflow

STEP 18 — Approve Goals API

```
@goal_bp.route('/approve-goal/<int:id>', methods=['PUT'])
def approve_goal(id):
    goal = Goal.query.get(id)

    goal.is_locked = True

    db.session.commit()

    return {
        'message': 'Goal approved and locked'
    }
```

STEP 19 — Reject Goal API

```
@goal_bp.route('/reject-goal/<int:id>', methods=['PUT'])
def reject_goal(id):
    return {
        'message': 'Goal returned for rework'
    }
```

11. Quarterly Check-ins

STEP 20 — Create Check-in Model

models/checkin_model.py

```
from app import db

class Checkin(db.Model):
    __tablename__ = 'checkins'

    id = db.Column(db.Integer, primary_key=True)
    goal_id = db.Column(db.Integer)
    quarter = db.Column(db.String(20))
    achievement = db.Column(db.Float)
    employee_comment = db.Column(db.Text)
    manager_comment = db.Column(db.Text)
    status = db.Column(db.String(50))
```

STEP 21 — Create Check-in API

```
@goal_bp.route('/checkin', methods=['POST'])
def create_checkin():
    data = request.json

    checkin = Checkin(
        goal_id=data['goal_id'],
        quarter=data['quarter'],
        achievement=data['achievement'],
        employee_comment=data['employee_comment'],
        status=data['status']
    )

    db.session.add(checkin)
    db.session.commit()

    return {'message': 'Check-in added'}
```

12. Progress Score Calculation

The problem statement defines formulas.

Min Type Formula

Higher is better.

Formula:

Achievement ÷ Target

Example:

$80 \div 100 = 0.8 = 80\%$

Max Type Formula

Lower is better.

Formula:

Target ÷ Achievement

STEP 22 — Implement Score Calculation

```
def calculate_progress(uom, target, achievement):  
  
    if uom == 'MIN':  
        return (achievement / target) * 100  
  
    elif uom == 'MAX':  
        return (target / achievement) * 100  
  
    elif uom == 'ZERO':  
        return 100 if achievement == 0 else 0  
  
    return 0
```

13. Audit Logs

STEP 23 — Track Goal Changes

Whenever a manager/admin edits a goal:

```
log = AuditLog(  
  user_id=user_id,  
  goal_id=goal.id,  
  action='Goal Updated',  
  old_value=str(old_data),  
  new_value=str(new_data)  
)
```

14. Frontend Development

STEP 24 — Install Frontend Dependencies

```
npm install axios  
npm install react-hook-form  
npm install lucide-react
```

STEP 25 — Install shadcn/ui

```
npx shadcn@latest init
```

STEP 26 — Frontend Folder Structure

```
frontend/  
├─ app/  
├─ components/  
├─ services/  
├─ employee/  
├─ manager/  
├─ admin/  
└─ lib/
```

15. Login Page

STEP 27 — Create Login UI

app/login/page.tsx

```
export default function LoginPage() {  
  return (  
    <div>  
      <h1>Login</h1>  
    </div>  
  )  
}
```

STEP 28 — Create API Service

services/api.ts

```
import axios from 'axios'  
  
const api = axios.create({  
  baseURL: 'http://127.0.0.1:5000'  
})  
  
export default api
```

16. Employee Dashboard

STEP 29 — Employee Dashboard Features

Show:

- Total goals
- Completed goals
- Pending goals
- Quarterly progress

STEP 30 — Create Goal Form

Fields:

- Thrust Area
 - Goal Title
 - Description
 - UoM
 - Target
 - Weightage
-

17. Manager Dashboard

STEP 31 — Manager Features

Manager should see:

- Pending approvals
 - Team goals
 - Check-ins
 - Team progress
-

STEP 32 — Approval Table

Each row:

- Employee name
 - Goal title
 - Weightage
 - Approve button
 - Reject button
-

18. Admin Dashboard

STEP 33 — Admin Features

Admin should manage:

- Users

- Goal cycles
 - Audit logs
 - Unlock goals
-

STEP 34 — Unlock Goal Feature

```
@goal_bp.route('/unlock-goal/<int:id>', methods=['PUT'])
def unlock_goal(id):
    goal = Goal.query.get(id)

    goal.is_locked = False

    db.session.commit()

    return {'message': 'Goal unlocked'}
```

19. Reporting Module

STEP 35 — Export CSV

Install:

```
pip install pandas
```

Export Logic

```
import pandas as pd
```

Convert goals/check-ins to dataframe.

Export CSV.

20. Completion Dashboard

STEP 36 — Dashboard Metrics

Show:

- Employees completed check-ins
 - Managers completed approvals
 - Pending reviews
-

21. Bonus Features

STEP 37 — Email Notifications

Use:

- Flask-Mail or
- SendGrid

Notify:

- Goal submitted
 - Goal approved
 - Check-in reminder
-

STEP 38 — Charts & Analytics

Use:

- Recharts

Charts:

- Quarterly progress
 - Completion rate
 - Goal distribution
-

STEP 39 — Teams/Slack Notifications

Optional bonus feature.

22. Deployment

STEP 40 — Deploy Database

Use:

- Supabase PostgreSQL
-

STEP 41 — Deploy Backend

Use:

- Render

Add:

```
Start Command:  
python app.py
```

STEP 42 — Deploy Frontend

Use:

- Vercel

Connect GitHub repository.

23. Demo Preparation

STEP 43 — Create Demo Accounts

Employee

```
employee@test.com
```

Manager

manager@test.com

Admin

admin@test.com

STEP 44 — Demo Flow

Employee Journey

- Login
 - Create goals
 - Submit goals
 - Update achievements
-

Manager Journey

- Login
 - Review goals
 - Approve/reject
 - Add comments
-

Admin Journey

- Unlock goals
 - View reports
 - View audit logs
-

24. Important Hackathon Advice

Focus First On:

1. Authentication
2. Goal CRUD

3. Validation Rules
 4. Approval Workflow
 5. Quarterly Check-ins
 6. Dashboard
-

Avoid Spending Too Much Time On:

- Fancy animations
 - Complex microservices
 - Over-engineering
-

Judges Usually Care Most About:

- End-to-end functionality
 - Stability
 - Smooth workflows
 - Clean UI
 - Working role-based access
-

25. Suggested Timeline

Day 1

Complete:

- Backend setup
 - Database
 - Authentication
 - Goal APIs
-

Day 2

Complete:

- Frontend dashboards
 - Approval workflow
 - Check-ins
-

Day 3

Complete:

- Reports
 - Audit logs
 - Deployment
 - Demo preparation
-

Final Result

By following this guide, you will have:

- A complete enterprise-style goal management portal
- Real role-based workflow
- Quarterly tracking system
- Manager approval system
- Audit-ready architecture
- Deployable production demo

This is more than enough for a strong hackathon submission.